Programmation orientée objet (UAA14)

Exercices : série 3

Objets d'apprentissage :

- ✔ Modéliser une logique de programmation orientée objet.
- ✔ Déclarer une classe.
- ✔ Instancier une classe.
- ✔ Utiliser les méthodes de l'objet instancié.
- ✔ Traduire un algorithme dans un langage de programmation.
- ✔ Commenter des lignes de code.
- ✔ Tester le programme conçu.
- ✔ Caractériser les attributs dans une classe (encapsulation).
- ✔ Caractériser les méthodes dans une classe (encapsulation).
- ✔ Décrire la création d'un constructeur.
- ✔ Extraire d'un cahier des charges les informations nécessaires à la programmation.
- ✔ Programmer en recourant aux instructions et types de données nécessaires au développement d'une application.
- ✔ Corriger un programme défaillant.
- ✔ Développer une classe sur la base d'un cahier des charges en respectant le paradigme de la programmation orientée objet.
- ✔ Programmer en recourant aux classes nécessaires au développement d'une application orientée objet.
- ✔ Améliorer un programme pour répondre à un besoin défini.

Exercice 1 : rencontre

On te demande de créer un programme PHP sur base du domaine d'application suivant :

Dans un tournoi de football, plusieurs équipes se rencontrent. Une rencontre oppose deux équipes : une équipe « locale » et une équipe « visiteur ». Au fur et à mesure du déroulement de la rencontre, des points sont donnés à chacune des équipes par un arbitre. Cet arbitre peut également pénaliser une équipe en lui attribuant des fautes.

Étape 1

Crée la classe `Rencontre` qui contient les v.i suivantes :

- `$nomLocaux` : le nom de l'équipe locale
- `$nomVisiteurs` : le nom de l'équipe visiteur
- `$pointsLocaux` : le nombre de points de l'équipe locale
- `$pointsVisiteurs` : le nombre de points de l'équipe visiteur
- `$nbFautesLocaux` : le nombre de fautes de l'équipe locale
- `$nbFautesVisiteurs` : le nombre de fautes de l'équipe visiteur

Prévois un constructeur permettant de créer des objets de type `Rencontre`.

Ajoute les méthodes suivantes :

- `vainqueur` : retourne le nom de l'équipe gagnante ou « Egalité »
- `équipeFairPlay` : retourne le nom de l'équipe qui a fait le moins de fautes ou « Egalité »
- `égalité` : retourne un booléen avec `true` si les équipes ont toutes les deux le même nombre de points et `false` sinon
- `présenterLocaux` : retourne une chaine de caractère qui présente l'équipe locale sous la forme « l'équipe locale `<nom de l'équipe>` »
- `présenterVisiteurs` : retourne une chaine de caractère qui présente l'équipe visiteur sous la forme « l'équipe visiteur `<nom de l'équipe>` »
- `ajouterPointsLocaux` : ajoute 1 point à l'équipe locale
- `ajouterPointsVisiteurs` : ajoute 1 point à l'équipe visiteur
- `ajouterFauteLocaux` : ajoute 1 faute à l'équipe locale
- `ajouterFauteVisiteur` : ajoute 1 faute à l'équipe visiteur

Étape 2

En utilisant la classe `Rencontre`,on te demande :

- de créer au moins un objet de type `Rencontre`
- d'ajouter des points et des fautes aux deux équipes d'une même rencontre (utilise les méthodes adéquates !)
- d'afficher à l'écran : la présentation des deux équipes, le nom de l'équipe victorieuse et le nom de l'équipe la plus fairplay
- de tester (en faisant appel à la méthode adéquate) s'il s'agit d'une rencontre où les équipes sont ex aequo

Étape 3

Ajoute une méthode `présenterAdversaire` qui retourne une chaine de caractères qui présente les deux équipes de la manière suivante :

« L'équipe locale `<nom de l'équipe locale>` reçoit l'équipe des visiteurs `<nom de l'équipe visiteur>` ».

Attention: tu dois obligatoirement utiliser les méthodes déjà existantes `présenterLocaux` et `présenterVisiteurs`.

Étape 4

Ajoute une méthode appelée `ajouterPoints` qui ajoute 1 point à l'équipe locale ou visiteur en fonction d'un caractère reçu en argument : 'L' pour l'équipe locale et 'V' pour l'équipe visiteur.

Attention: tu dois obligatoirement utiliser les méthodes déjà existantes `ajouterPointsLocaux` et `ajouterPointsVisiteurs`.

Étape 5

Adapte le constructeur de la classe `Rencontre` de telle manière que l'on puisse créer facilement une rencontre où les deux équipes n'ont pas encore de point ni de faute (c'est-à-dire 0 partout).

Étape 6

Prévois, toujours dans la classe `Rencontre`, une méthode nommée `présenter` qui décrit une rencontre de la manière suivante :

« L'équipe locale `<nom de l'équipe locale>` reçoit l'équipe visiteur `<nom de l'équipe visiteur>`. L'équipe victorieuse est `<nom de l'équipe gagnante>`.»

Étape 7

Adapte les méthodes `ajouterPointsLocaux` et `ajouterPointsVisiteurs` afin qu'elles reçoivent en argument un entier facultatif représentant le nombre de points à ajouter à l'équipe (donc 1 par défaut).

Exercice 2 : de la table à l'objet

Dans une application web, les données ne sont pas écrites dans le code : elles sont rangées dans une base de données. Le programme les reçoit **ligne par ligne**, sous la forme de tableaux associatifs, et son tout premier travail consiste à en faire des objets.

Cet exercice reproduit cette mécanique, sans base de données.

Étape 1

Voici la structure de la table `membre` d'une application de pronostics :

```
Colonne   Type      Remarque
-------   ----      ------------------------------
Pseudo    chaine    identifie le membre
Email     chaine
Points    entier    vaut 0 a l'inscription
Admin     booleen   vaut faux a l'inscription
```

Crée la classe `Membre` : **une variable d'instance privée par colonne**, en conservant exactement les mêmes noms. Prévois un `__get` générique.

Étape 2

Écris le constructeur. Les deux dernières colonnes ont une valeur connue d'avance au moment de l'inscription : fais en sorte qu'on puisse créer un membre sans avoir à les préciser.

```php
$zoe = new Membre("zoe", "zoe@mail.be");            // 0 point, pas admin
$sam = new Membre("sam", "sam@mail.be", 63, true);
```

Étape 3

Voici trois lignes telles que le programme les recevrait de la base de données :

```php
$lignes = array(
    array("Pseudo" => "zoe",   "Email" => "zoe@mail.be",   "Points" => 42,
"Admin" => false),
    array("Pseudo" => "malik", "Email" => "malik@mail.be", "Points" => 17,
"Admin" => false),
    array("Pseudo" => "sam",   "Email" => "sam@mail.be",   "Points" => 63,
"Admin" => true)
);
```

Écris le code qui parcourt `$lignes` et construit un tableau `$membres` contenant trois objets de type `Membre`.

Étape 4

Parcours `$membres` et affiche le pseudo et le nombre de points de chacun.

Étape 5

Ajoute à la classe `Membre` les deux méthodes suivantes :

- `estAdmin` : retourne un booléen
- `ajouterPoints` : reçoit un nombre de points en argument et les ajoute au total du membre

Utilise-les : ajoute 10 points à zoe, puis affiche uniquement les pseudos des membres administrateurs.

Étape 6

En réalité, la table `membre` possède une colonne de plus :

```
Colonne   Type      Remarque
-------   ------    ------------------------------------------------
Id        entier    attribué automatiquement par la base de données
```

Quand un visiteur s'inscrit, le programme construit l'objet `Membre` **avant** de l'envoyer à la base de données. À cet instant, l'identifiant n'existe pas encore : c'est la base qui l'attribuera.

Ajoute une variable d'instance `Id` à ta classe, ainsi que l'argument correspondant **en dernière position** dans le constructeur, avec `null` comme valeur par défaut. Vérifie ensuite que les deux situations suivantes fonctionnent :

```php
$nouveau = new Membre("lea", "lea@mail.be");
// membre pas encore enregistre : son Id vaut null

$relu = new Membre("zoe", "zoe@mail.be", 42, false, 7);
// membre relu depuis la base : il a recu l'identifiant 7
```

Réponds enfin aux deux questions suivantes :

- pourquoi `null` plutôt que 0 comme valeur par défaut ?
- pourquoi cet argument doit-il obligatoirement être le dernier de la liste ?

À retenir : la même classe sert dans les deux sens — pour envoyer une donnée vers la base et pour la relire ensuite. C'est l'argument facultatif qui rend cela possible.

Exercice 3 : le classement

Règle du jeu

À partir d'ici, on ne te donne plus d'étapes. Ni la liste des variables d'instance, ni celle des méthodes : c'est à toi de les décider. Tu ne reçois que la situation à modéliser, quelques exigences, et le résultat attendu à l'écran.

La situation

Voici, telles quelles, les lignes que la base de données renvoie pour la table `participant` d'un concours de pronostics :

```php
$lignes = array(
    array("Id" => 3, "Pseudo" => "malik", "Bons" => 12, "Total" => 20),
    array("Id" => 1, "Pseudo" => "zoe",   "Bons" => 17, "Total" => 20),
    array("Id" => 7, "Pseudo" => "sam",   "Bons" => 9,  "Total" => 20)
);
```

« Bons » est le nombre de pronostics corrects, « Total » le nombre de pronostics joués.

Les exigences

- une classe, une ligne de la table ;
- le pourcentage de réussite ne se stocke pas ;
- un participant qui vient d'être créé, et qui n'est pas encore enregistré, n'a pas d'identifiant — ton programme doit en créer un et le signaler, comme dans la dernière ligne attendue ;
- ni la recherche du meilleur participant ni le calcul de la moyenne ne sont des méthodes de ta classe.

Le résultat attendu

Affiche les participants dans l'ordre où la base les renvoie.

```
--- RESULTATS ---
malik  12/20  (60 %)
zoe    17/20  (85 %)
sam     9/20  (45 %)

Meilleur participant : zoe (85 %)
Moyenne du concours  : 63.3 %

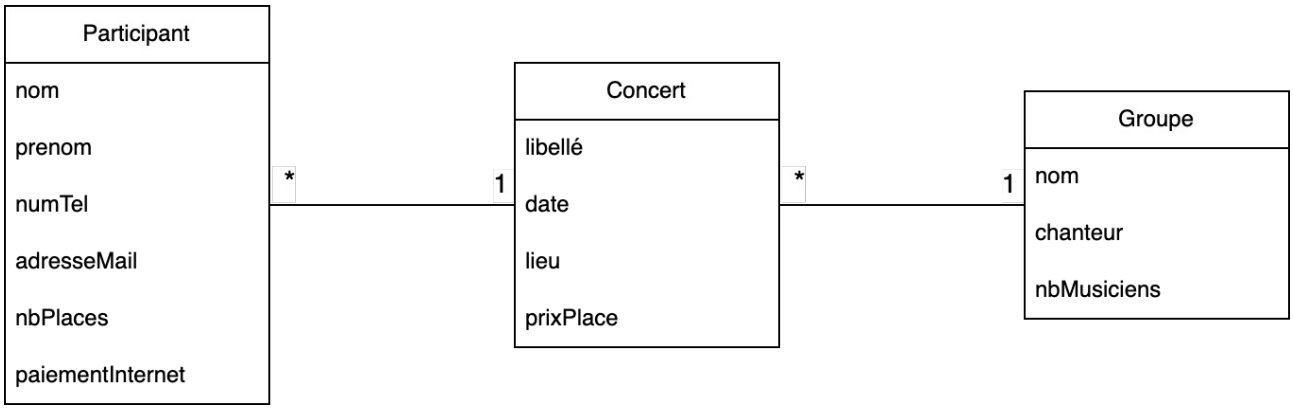
Nouvelle inscription : lea — identifiant pas encore attribué
```

Pour vérifier

- combien de variables d'instance ta classe possède-t-elle, et pourquoi exactement ce nombre ?
- où as-tu écrit le code qui transforme les lignes en objets ? si l'application comptait dix pages, combien de fois faudrait-il l'écrire ? que proposerais-tu ?
- quel argument de ton constructeur vaut `null` par défaut, et à quel moment recevra-t-il une vraie valeur ?

Exercice 4 : concert

L'objectif de cet exercice est de modéliser la situation suivante en PHP :



Étape 1

Crée une classe intitulée `Concert` permettant de créer des concerts.

Un concert est caractérisé par :

- un libellé
- une date
- un lieu
- un prix d'entrée

Étape 2

Crée un constructeur permettant de créer des concerts.

Crée ensuite au moins deux concerts.

Étape 3

Dans la classe `Concert`, crée une méthode `__toString()` qui renvoie une chaine de caractère présentant le concert.

Affiche ensuite la présentation des concerts créés.

Étape 4

Affiche la date et l'endroit des concerts créés (ajoute les méthodes nécessaires dans la classe `Concert`).

Étape 5

Crée une classe intitulée `Participant` permettant de créer des personnes qui participent à un concert.

Un participant ne peut s'inscrire qu'à un seul concert. Bien évidemment, un concert peut accueillir plusieurs participants.

Un participant est caractérisé par :

- un nom
- un prénom
- un numéro de téléphone
- une adresse mail
- le nombre de places achetées
- le fait qu'il souhaite payer par internet ou non

Étape 6

Prévois un constructeur permettant de créer des participants.

Crée ensuite au moins trois participants.

Étape 7

Dans la classe `Participant`, prévois une méthode nommée `identité` retournant l'identité (nom / prénom) d'un participant.

Étape 8

Prévois une méthode `__toString()` qui retourne une chaîne de caractère décrivant un participant de la manière suivante :

« `<identité du participant>` participe au concert `<présentation du concert>`.»

Attention : pour implémenter cette méthode, tu dois utiliser la méthode `identité` ainsi que la méthode `__toString()` de la classe `Concert` !

Affiche ensuite la présentation des participants créés.

Étape 9

Dans la classe `Participant`, prévois les méthodes permettant :

- de modifier (augmenter / diminuer) le nombre de places achetées sur base d'un entier reçu en argument
- de calculer le montant à payer

Ensuite, (en utilisant les méthodes adéquates !):

- augmente le nombre d'inscrits pour un des participants
- diminue le nombre d'inscrits pour un des autres participants
- affiche le montant total à payer pour chacun des participants
- affiche l'identité des participants

Étape 10

Affiche :

- le libellé du concert choisi par le premier participant créé
- la date du concert choisi par le deuxième participant créé
- le prix par personne du concert choisi par le troisième participant créé

Étape 11

Crée une classe nommée `Groupe` permettant de gérer des groupes musicaux.

Un groupe est caractérisé par :

- un nom
- le nom du chanteur
- le nombre de musiciens

Étape 12

Crée un constructeur pour la classe `Groupe` permettant des créer des groupes musicaux.

Crée quelques groupes musicaux de ton choix.

Étape 13

Prévois la méthode `__toString()` qui retourne une chaine de caractère décrivant un groupe.

Affiche la présentation des groupes créés.

Étape 14

Modifie la classe `Concert` de telle manière que l'on puisse retrouver le groupe qui se produit lors du concert. N'oublie pas d'adapter le constructeur !

Adapte les instructions de création des objets de type `Concert`.

Étape 15

Adapte la méthode `présenter` de la classe `Concert` afin que la présentation reprenne la présentation du groupe qui se produit lors de ce concert.

Vérifie que la description des concerts est correcte (elle doit reprendre la description des groupes !).

Étape 16

Enfin :

- affiche le nom du groupe se produisant au concert choisi par le premier participant créé
- affiche le nombre de musiciens du groupe se produisant au concert choisi par le deuxième participant créé
- affiche le nom du chanteur se produisant au concert choisi par le troisième participant créé

Références

Les présents exercices ont été élaborés à l'aide des ressources suivantes :

- https://www.pierre-giraud.com/php-mysql-apprendre-coder-cours/introduction-programmation-orientee-objet/
- Cours de « Développement d'applications WEB », Hénallux (2019)